

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра САПР

ОТЧЕТ

по лабораторной работе №5

по дисциплине «Компьютерная графика»

**ТЕМА: «Исследование алгоритмов выявления видимости сложных
сцен»**

Студенты гр. 3311

Аршин А.Д.
Баймухамедов
Р.Р.
Пасечный Л.В.

Преподаватель

Колев Г. Ю.

Санкт-Петербург

2025

Цель работы:

Целью данной работы является разработка программного обеспечения для визуализации и управления видимостью совокупности произвольных многогранников с использованием алгоритма сортировки граней по Z-координате (painter's algorithm).

Для достижения поставленной цели необходимо решить следующие задачи:

1. Реализовать математическую модель многогранников с возможностью хранения вершин и граней
2. Разработать функции для создания различных типов многогранников (куб, тетраэдр)
3. Реализовать механизм поворота многогранников вокруг осей X и Y с использованием матриц поворота
4. Внедрить алгоритм определения лицевых и обратных граней через скалярное произведение нормали и вектора наблюдения
5. Реализовать алгоритм сортировки граней по глубине (Z-координате) для корректного определения видимости
6. Создать интерактивный интерфейс с слайдерами для динамического управления поворотом сцен

Задание (вариант 3):

Обеспечить реализацию видимости совокупности произвольных многогранников на основе алгоритма сортировки граней по Z координате

Теоретическая основа:

Матрица поворота в двумерном пространстве [\[править | править код \]](#)

В двумерном пространстве поворот можно описать одним углом θ со следующей матрицей [линейного преобразования в декартовой системе координат](#):

$$M(\theta) = \begin{pmatrix} \cos \theta & \mp \sin \theta \\ \pm \sin \theta & \cos \theta \end{pmatrix}$$

или

$$M(\theta) = \exp\left(\theta \cdot \begin{bmatrix} 0 & \mp 1 \\ \pm 1 & 0 \end{bmatrix}\right).$$

Поворот выполняется путём умножения матрицы поворота на [вектор-столбец](#), описывающий вращаемую точку:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & \mp \sin \theta \\ \pm \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}.$$

Координаты (x', y') в результате поворота точки (x, y) имеют вид:

$$\begin{aligned} x' &= x \cos \theta \mp y \sin \theta, \\ y' &= \pm x \sin \theta + y \cos \theta. \end{aligned}$$

Конкретные знаки в формулах зависят от того, является ли система координат правосторонней или левосторонней, и выполняется ли вращение по или против часовой стрелки. Верхний знак указан для обычного соглашения: правосторонняя система координат и положительное направление вращения против часовой стрелки (тот же знак верен для левосторонней координатной системы при выборе положительного направления вращения по часовой стрелке; в оставшихся двух комбинациях — нижний знак).

12 Литература
13 Ссылки

Рис. 1

Билинейная поверхность конструируется из четырех угловых точек единичного квадрата в параметрическом пространстве, т.е. из точек $P(0,0), P(0,1), P(1,0), P(1,1)$. Любая точка на поверхности определяется линейной интерполяцией между противоположными границами единичного квадрата. Любая точка внутри параметрического квадрата задается уравнением: $Q(u,w)=P(0,0)(1-u)(1-w)+P(0,1)(1-w)+P(1,1)uw/P$. Необходимо, чтобы интерполируемая поверхность удовлетворяла исходным данным. В этом случае легко проверить, что угловые точки принадлежат этой поверхности, т.е. $Q(0,0)=P(0,0)$ и т.д. Если координатные векторы четырех точек, определяющих билинейную поверхность, заданы в трехмерном объектном пространстве, то будет трехмерна и билинейная поверхность, получаемая в результате отображения параметрического пространства в объектное. Если четыре определяющие точки не лежат в одной плоскости, то и билинейная поверхность также не лежит ни в какой плоскости. Сопряжение углов дает билинейную поверхность.

Сопряжение граничных кривых произвольной формы дает поверхность, называемую лоскутом Куна. Слово «лоскут» указывает на то, что описываемая поверхность представляет собой сегмент, соответствующий значениям параметров. Комбинирование лоскутов позволяет образовать поверхность произвольной формы и размера. Формула: $P(u,v)=(1-u)P_0(v)+uP_1(v)+(1-v)Q_0(u)+vQ_1(u)-(1-u)(1-v)P_0,0-u(1-v)P_1,0-(1-u)vP_0,1-uvP_1,1$

Бикубический лоскут (bicubic patch) - это поверхность, определяемая полиномиальным уравнением третьего порядка по параметрам u и v .

$$P(u,v) = \sum_{i=0}^3 \sum_{j=0}^3 a_{i,j} u^i v^j \quad (0 \leq u \leq 1, 0 \leq v \leq 1).$$

13. Конические сечения кривые. Кривая Безье, b-сплайн

$B(t) = \sum_{i=0}^n P_i b_{i,n}(t), \quad 0 < t < 1$ $B(t) = (1-t)P_0 + tP_1 \quad t \in [0,1]$ кривая Безье — параметрическая кривая, задаваемая выражением: Линейные кривые При $n = 1$ кривая представляет собой отрезок прямой линии, опорные точки P_0 и P_1 определяют его начало и конец. Кривая задается уравнением:

Квадратичная кривая Безье ($n = 2$) задается 3-мя опорными точками: P_0, P_1 и P_2 .

$$B(t) = (1-t)^2 P_0 + 2t(1-t)P_1 + t^2 P_2, \quad t \in [0, 1]$$

Квадратичные кривые Безье в составе сплайнов используются для описания формы символов в шрифтах TrueType и в SWF файлах.

Кубические кривые В параметрической форме кубическая кривая Безье ($n = 3$) описывается следующим уравнением:

$$B(t) = (1-t)^3 P_0 + 3t(1-t)^2 P_1 + 3t^2(1-t)P_2 + t^3 P_3, \quad t \in [0, 1]$$

Благодаря простоте задания и манипуляции, кривые Безье нашли широкое применение в компьютерной графике для моделирования гладких линий. Кривая целиком лежит в выпуклой оболочке своих опорных точек. Это свойство кривых Безье с одной стороны значительно облегчает задачу нахождения точек пересечения кривых, а с другой стороны позволяет осуществлять интуитивно понятное управление параметрами кривой в графическом интерфейсе с помощью её опорных точек.

В вычислительной математике В-сплайном называют сплайн-функцию, имеющую наименьший носитель для заданной степени, порядка гладкости и разбиения области определения. Фундаментальная теорема устанавливает, что любая сплайн-функция для заданной степени, гладкости и области определения может быть представлена как линейная комбинация В-сплайнов той же степени и гладкости на той же области определения. Термин В-сплайн был введен И. Шёнбергом и является сокращением от словосочетания «базисный сплайн». [2] В-сплайны могут быть вычислены с помощью алгоритма де Бора, обладающего устойчивостью. В системах автоматизированного проектирования и компьютерной графике термин В-сплайн часто описывает сплайн-кривую, которая задана сплайн-функциями, выраженными линейными комбинациями В-сплайнов.

Рис. 2

Выполнение лабораторной работы

В качестве языка программирования для выполнения данной лабораторной работы выберем Python. Для быстрого запуска достаточно иметь установленный Python 3.

Демонстрация (<https://disk.yandex.ru/i/DEmNPErOonhd9w>)

Интерфейс созданного приложения:

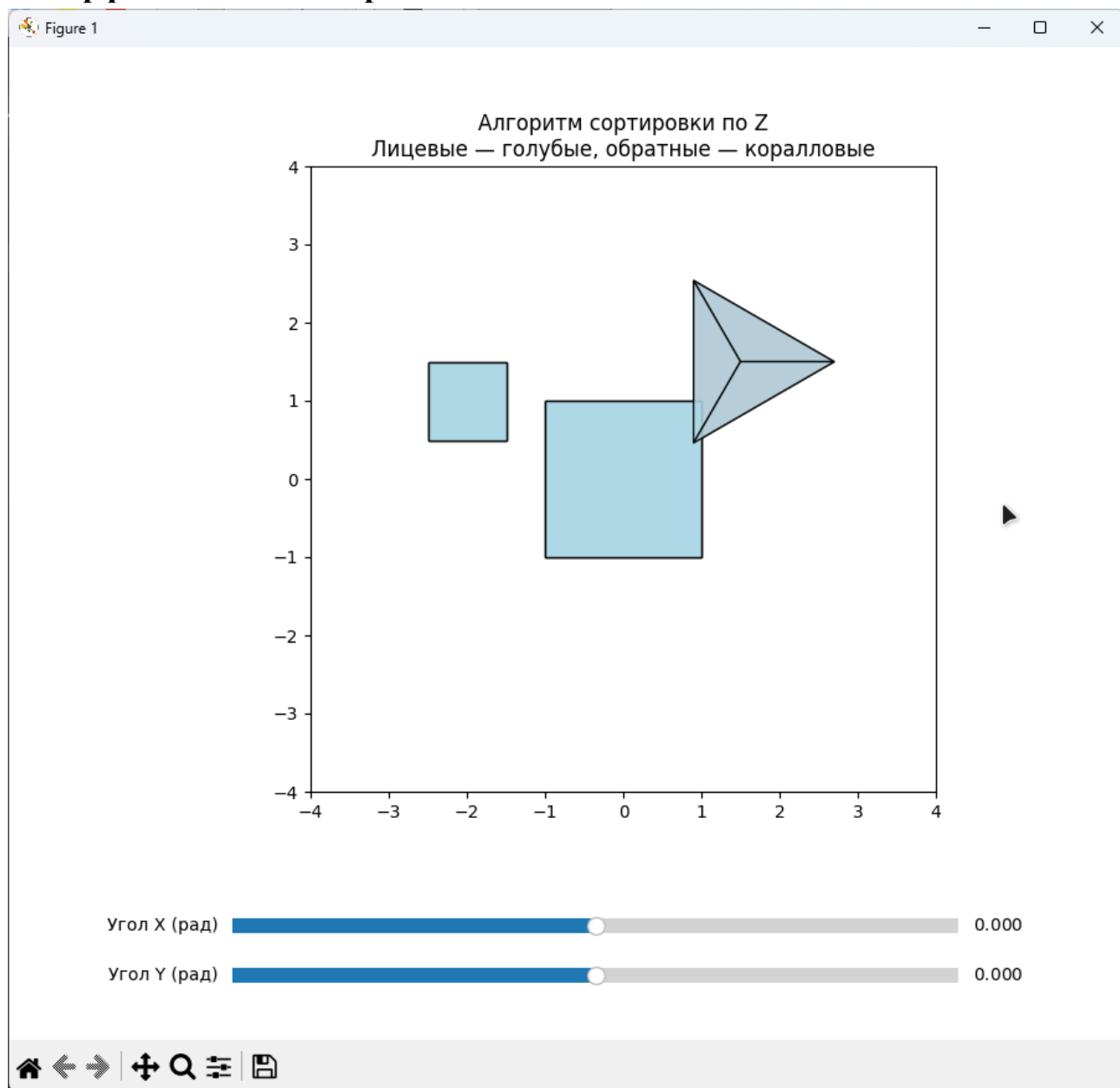


Рис. 3 Готовое приложение

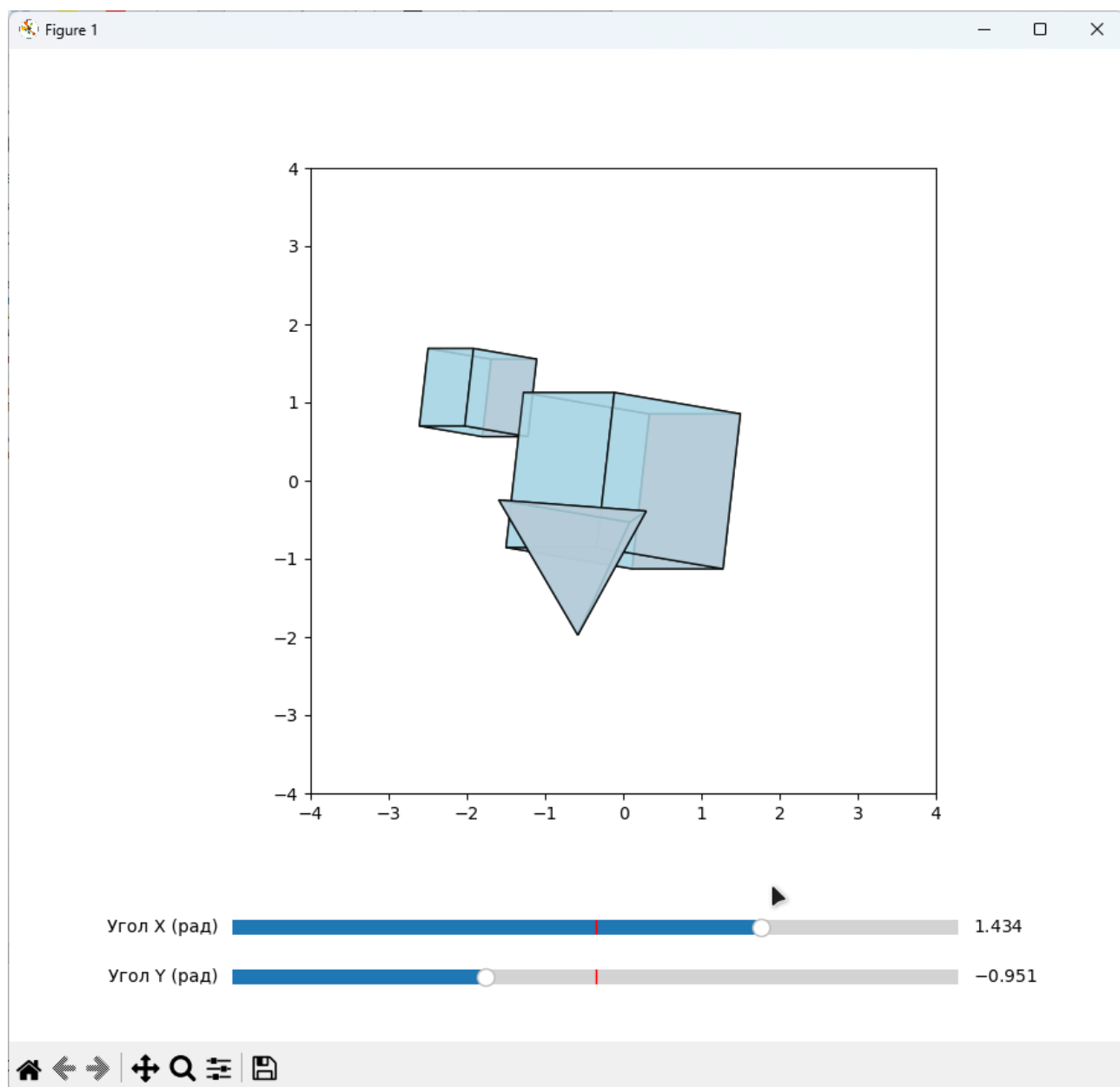


Рис. 4 показ работы с настройкой углов

Вывод:

В ходе выполнения работы был успешно реализован и протестирован алгоритм сортировки граней по Z-координате для обеспечения видимости совокупности многогранников. Анализ результатов позволяет сделать следующие выводы:

1. **Эффективность алгоритма сортировки:** Алгоритм (сортировка по глубине) доказал свою эффективность для сцен со средним количеством объектов. Грани корректно сортируются от дальних к ближним, что обеспечивает правильное перекрытие при отрисовке.
2. **Корректность определения видимости граней:** Механизм определения лицевых и обратных граней через скалярное произведение нормали и вектора наблюдения работает корректно. Разделение цветов (голубой для лицевых, коралловый для обратных) наглядно демонстрирует ориентацию многогранников в пространстве.

3. **Гибкость системы преобразований:** Использование матриц поворота обеспечивает точные и плавные преобразования многогранников. Комбинирование поворотов вокруг осей X и Y позволяет всесторонне изучать геометрию сцены.
4. **Интерактивность и наглядность:** Реализованный интерфейс с слайдерами предоставляет удобный инструмент для интерактивного исследования трехмерной сцены. Динамическое обновление визуализации при изменении углов поворота демонстрирует реальную работу алгоритма сортировки.
5. **Масштабируемость архитектуры:** Класс Polyhedron обеспечивает гибкую структуру для работы с различными типами многогранников. Система легко расширяема для добавления новых фигур и преобразований.
6. **Ограничения алгоритма:** Алгоритм сортировки по Z показывает свои ограничения при работе со сложными перекрывающимися объектами, где требуется более точное определение видимости на пиксельном уровне (z-buffer). Однако для демонстрационных целей он полностью справляется с поставленной задачей.

Приложение 1: Листинг программы

```
import matplotlib.pyplot as plt
import numpy as np
import math
from matplotlib.widgets import Slider

# описание многогранника
# vertices – список координат вершин (x, y, z).
# faces – список граней, каждая грань – это список индексов вершин.
# color_front / color_back – цвета для лицевых и обратных граней (чтобы видеть,
какая сторона "смотрит на нас").
class Polyhedron:
    def __init__(self, vertices, faces, color_front='lightblue',
color_back='lightcoral'):
        self.vertices = np.array(vertices)
        self.faces = faces
        self.color_front = color_front
        self.color_back = color_back
    # Возвращает центр грани
    def get_face_center(self, face):
        return np.mean(self.vertices[face], axis=0)

    # вычисляет нормаль к грани
    # использует векторное произведение двух ребер(v1 v2)
    # нормализует вектор(делит на длину), чтобы он был единичным
    # если грань вырождена(меньше 3 точек), то возвращает заглушку 0 0 1
    def get_face_normal(self, face):
        pts = self.vertices[face]
        if len(pts) < 3:
            return np.array([0, 0, 1])
        v1 = pts[1] - pts[0]
```

```

        v2 = pts[2] - pts[0]
        normal = np.cross(v1, v2)
        norm = np.linalg.norm(normal)
        return normal / norm if norm != 0 else np.array([0, 0, 1])

# Возвращает матрицу поворота вокруг X на заданный угол
def rotation_matrix_x(angle):
    c, s = math.cos(angle), math.sin(angle)
    return np.array([[1, 0, 0],
                    [0, c, -s],
                    [0, s, c]])

# Аналогичный поворот только вокруг Y
def rotation_matrix_y(angle):
    c, s = math.cos(angle), math.sin(angle)
    return np.array([[c, 0, s],
                    [0, 1, 0],
                    [-s, 0, c]])

# Создает куб с 8-ю вершинами, 6-ю гранями
def create_cube(center=(0, 0, 0), size=1.0):
    cx, cy, cz = center
    s = size / 2
    v = [
        [cx - s, cy - s, cz - s],
        [cx + s, cy - s, cz - s],
        [cx + s, cy + s, cz - s],
        [cx - s, cy + s, cz - s],
        [cx - s, cy - s, cz + s],
        [cx + s, cy - s, cz + s],
        [cx + s, cy + s, cz + s],
        [cx - s, cy + s, cz + s],
    ]
    f = [[0,1,2,3], [4,5,6,7], [0,1,5,4], [2,3,7,6], [0,3,7,4], [1,2,6,5]]
    return Polyhedron(v, f)

# Создание тетраэдра с 4-мя вершинами и 4-мя гранями
def create_tetrahedron(center=(0, 0, 0), size=1.0):
    cx, cy, cz = center
    s = size
    v = [
        [cx, cy, cz + s],
        [cx + s, cy, cz - s/3],
        [cx - s/2, cy + s*math.sqrt(3)/2, cz - s/3],
        [cx - s/2, cy - s*math.sqrt(3)/2, cz - s/3],
    ]
    f = [[0,1,2], [0,2,3], [0,3,1], [1,3,2]]
    return Polyhedron(v, f)

# ax — оси matplotlib.
# polyhedra — список объектов Polyhedron.
# view_dir — направление взгляда (по умолчанию — вдоль оси Z, от нас к экрану).
def render_with_z_sort(ax, polyhedra, view_dir=np.array([0, 0, -1])):
    # для каждой грани находим центр и нормаль

```

```

all_faces = []
for poly in polyhedra:
    for face in poly.faces:
        center = poly.get_face_center(face)
        normal = poly.get_face_normal(face)
        # скалярное произведение показывает смотрит ли грань на нас
        # если больше 0 -->лицевая-->color_front
        # иначе-->обратная-->color_back
        dot = np.dot(normal, -view_dir)
        color = poly.color_front if dot > 0 else poly.color_back
        # берем только x и y
        projected = poly.vertices[face][:, :2]
        # глубина по зед, для сортировки
        z_depth = center[2]
        #сортируем грани от самых дальних к самым ближним
        # это замена z-buffer, сначала рисуем дальние потом ближние - они
        # перекроют
        all_faces.append((z_depth, projected, color))
all_faces.sort(key=lambda x: x[0]) # сортировка по Z (дальние → ближние)

# очищаем оси и задаем пределы отображения
ax.clear()
ax.set_xlim(-4, 4)
ax.set_ylim(-4, 4)
ax.set_aspect('equal')

# рисуем каждую грань как 2-д полигон с цветом и черной обводкой
for _, verts, color in all_faces:
    poly_patch = plt.Polygon(verts, closed=True, facecolor=color,
                             edgecolor='k', alpha=0.85)
    ax.add_patch(poly_patch)

# ----- Основная программа -----
# Исходные объекты (без поворота)
# три объекта создаются
base_objects = [
    create_cube(center=(0, 0, 0), size=2),
    create_tetrahedron(center=(1.5, 1.5, 1), size=1.2),
    create_cube(center=(-2, 1, -1), size=1),
]

# Создаём фигуру и оси
fig, ax = plt.subplots(figsize=(9, 8))
plt.subplots_adjust(left=0.2, bottom=0.25)

# Начальные углы и место для слайдеров
angle_x0 = 0.0
angle_y0 = 0.0

# Применяем начальный поворот и рисуем
# сначала поворачиваем вокруг X , а потом Y
# для каждого создается копия и поворачиваются все вершины
def apply_rotation_and_render(angle_x, angle_y):
    R = rotation_matrix_y(angle_y) @ rotation_matrix_x(angle_x)

```



```

    current_objects = []
    for obj in base_objects:
        new_obj = Polyhedron(obj.vertices.copy(), obj.faces, obj.color_front,
obj.color_back)
        new_obj.vertices = (R @ new_obj.vertices.T).T
        current_objects.append(new_obj)
    # отрисовываем с сортировкой
    render_with_z_sort(ax, current_objects)

# рисуем начальное состояние
apply_rotation_and_render(angle_x0, angle_y0)

# Создаём слайдеры
ax_angle_x = plt.axes([0.2, 0.1, 0.65, 0.03])
ax_angle_y = plt.axes([0.2, 0.05, 0.65, 0.03])

# от -пи до пи
slider_x = Slider(ax_angle_x, 'Угол X (рад)', -np.pi, np.pi, valinit=angle_x0)
slider_y = Slider(ax_angle_y, 'Угол Y (рад)', -np.pi, np.pi, valinit=angle_y0)

# Обработчики изменений пересчитываем поворот и перерисовываем
def update(val):
    apply_rotation_and_render(slider_x.val, slider_y.val)
    fig.canvas.draw_idle()

# привязываем функцию update к изменениям слайдера
slider_x.on_changed(update)
slider_y.on_changed(update)

# Подписи
ax.set_title("Алгоритм сортировки по Z\nлицевые – голубые, обратные – коралловые",
fontsize=12)

plt.show()

```